CS301: Computability and Complexity Theory (CC)

Lecture 1: Intro, Recap and The Church-Turing Thesis

Dumitru Bogdan

Faculty of Computer Science
University of Bucharest

October 4, 2024

Table of contents

1. What is it about?
2. Administrative details
3. Recap CS112/LFA
4. Context setup
5. Turing Machines
6. Variants of Turing Machines

Section 1

What is it about?

Complexity Theory

Welcome to the almost fascinating world of Complexity Theory! This course will take you on a journey through the fundamental concepts and principles that underlie the study of computational problems and their inherent difficulty. We'll also explore how these concepts apply to cutting-edge fields like Artificial Intelligence (AI) and Machine Learning (ML).

Complexity Theory is a branch of theoretical computer science that focuses on classifying computational problems according to their inherent difficulty and relating these classes to each other. It provides a framework for understanding the limitations and possibilities of efficient computation, which is crucial in today's AI-driven world.

Key topics we'll explore

- Computational Models: Turing machines and other models of computation
- Time and Space Complexity: Analyzing the resources required by algorithms
- Complexity Classes: P, NP, PSPACE, and their relationships
- NP-completeness: Understanding the most challenging problems in NP
- Reduction Techniques: Proving problems are in specific complexity classes
- Complexity in AI and Machine Learning: Computational aspects of learning algorithms and AI systems

Why study complexity theory?

1. Complexity theory provides a rigorous mathematical framework for understanding computation, forming the backbone of theoretical computer science
2. You'll develop analytical thinking and problem-solving skills that are valuable in various areas of computer science and beyond
3. Understanding complexity helps in designing efficient algorithms and recognizing when to seek alternative approaches for intractable problems
4. Complexity theory has practical implications in cryptography, artificial intelligence, and optimization problems encountered in industry
5. Complexity theory provides crucial insights into the limitations and capabilities of AI and ML algorithms, helping us understand issues like computational learning theory, PAC learning, and the complexity of neural networks

Section 2

Administrative details

Team members

- Dumitru Bogdan - lectures and seminars
  bogdan.dumitru1@unibuc.ro
  bogdan.dumitru@fmi.unibuc.ro

Textbook and course materials

- Textbook: Michael Sipser - Introduction to Theory of Computation. (3nd edition recommended).
- Course materials: teams
  - lecture slides available after class
  - exercises/problems and solutions in seminars only
- Interaction: email, teams (chanel code is 3oug4gb)

Grading

The following components will contribute the indicated points to your grade:

1. Seminar activity - 30 points
2. Homework (coding) - 20 points
3. Final Exam - 100 points

To get a 10 you need 100 points, for a 5 you need 50 points.

Section 3

Recap CS112/LFA

DFA definition

Definition

A finite automaton is 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma \to Q$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states

NFA definition

Definition

A nondeterministic finite automaton is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states
2. Σ is a finite alphabet
3. $\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accepted states

We denote $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\mathcal{P}(Q)$ as the power set of Q.

RegExp definition

Definition

We say that R is a regular expression if R is:

1. a for some a in the alphabet Σ
2. ϵ
3. $\emptyset$
4. $(R_1 \cup R_2)$ where R_1 and R_2 are regular expressions
5. $(R_1 \circ R_2)$ where R_1 and R_2 are regular expressions, or
6. (R_1^*) where R_1 is a regular expression

In items 1 and 2, the regular expressions a and ϵ represent the languages $\{a\}$ and $\{\epsilon\}$, respectively. In item 3, the regular expression $\emptyset$ represents the empty language. In items 4, 5, and 6, the expressions represent the languages obtained by taking the union or concatenation of the languages R_1 and R_2, or the star of the language R_1, respectively.

CFG definition

Definition

A **context-free grammar** is a 4-tuple (V, Σ, R, S) where:

- V is a finite set called the **variables**
- Σ is a finite set, disjoint from V called the **terminals**
- R is a finite set of rules, with each rule being a variable and a string of variables and terminals
- $S \in V$ is the start variable

PDA definition

Definition

A pushdown automaton is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where Q, Σ, Γ and F are all finite sets, and

1. Q is the set of states
2. Σ is the input alphabet
3. Γ is the stack alphabet
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function
5. $q_0 \in Q$ is the start state
6. $F \in Q$ is the set of accept states

Section 4

Context setup

Context setup

Corresponding to Sipser 3.1

Context setup

- Previously we have presented several models of computing devices: DFA, PDA, CFG and others
- We have shown that some very simple tasks are beyond the capabilities of these models
- We now focus on more powerful model **Turing Machines**
- A Turing machine can do everything that a real computer can do

Section 5

Turing Machines

How it looks

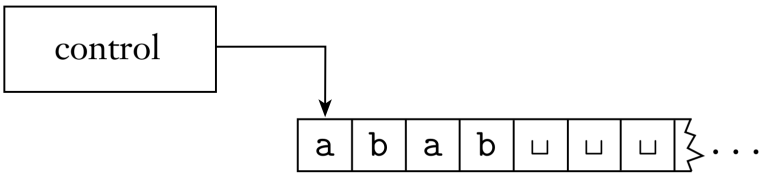


Figure: Schematic of a Turing machine

How it works

- The Turing machine (TM) model uses an infinite tape as its unlimited memory
- It has a tape head that can read and write symbols and move around on the tape
- Initially the tape contains only the input string and is blank everywhere else
- If the machine needs to store information, it may write this information on the tape
- To read the information that it has written, the machine can move its head back over it
- The machine continues computing until it decides to produce an output
- The outputs accept and reject are obtained by entering designated accepting and rejecting states
- If it doesn't enter an accepting or a rejecting state, it will go on forever, never halting

Differences between TM and FA

1. A Turing machine can both write on the tape and read from it
2. The read–write head can move both to the left and to the right
3. The tape is infinite
4. The special states for rejecting and accepting take effect immediately

Example

- We want to create M_1 for testing membership in the language $B = \{w\#w | w \in \{0,1\}^*\}$
- We want M_1 to accept if its input is a member of B and to reject otherwise
- Assume a very long input string, too long to be remembered, but you are allowed to move back and forth over the input and make marks on it
- The obvious strategy is to zig-zag to the corresponding places on the two sides of the # and determine whether they match
- It makes multiple passes over the input string with the read–write head
- On each pass it matches one of the characters on each side of the # symbol
- To keep track of which symbols have been checked already, M_1 crosses off each symbol as it is examined
- If it crosses off all the symbols, that means that everything matched successfully, and M_1 goes into an accept state
- If it discovers a mismatch, it enters a reject state

Example

M_1 = On input string w:

1. Zig-zag across the tape to corresponding positions on either side of the # symbol to check whether these positions contain the same symbol. If they do not, or if no # is found, reject. Cross off symbols as they are checked to keep track of which symbols correspond
2. When all symbols to the left of the # have been crossed off, check for any remaining symbols to the right of the #. If any symbols remain, reject; otherwise, accept

Example

The below figure contains several nonconsecutive snapshots of M_1's tape after it is started on input 011000#011000:

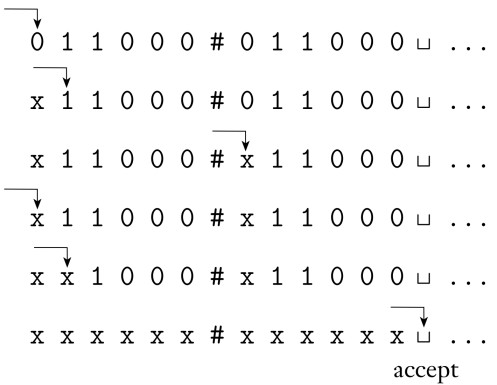


Figure: Snapshots of Turing machine M_1

Delta function

- The heart of the definition of a TM is the transition function δ because it tells us how the machine gets from one step to the next
- For a TM δ takes the form $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- When the TM is in a certain state q and the head is over a tape square containing a symbol a, and if $\delta(q, a) = (r, b, L)$, the machine writes the symbol b replacing the a, and goes to state r
- The third component is either L or R and indicates whether the head moves to the left or right after writing. In this case, the L indicates a move to the left.

Formal definition

Definition

A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ where Q, Σ, Γ are all finite sets and

1. Q is the set of states
2. Σ is the input alphabet not containing the *blank symbol* $\sqcup$
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function
5. $q_0 \in Q$ is the start state
6. $q_{accept} \in Q$ is the accept state
7. $q_{reject} \in Q$ is the reject state and $q_{accept} \neq q_{reject}$

TM diagram

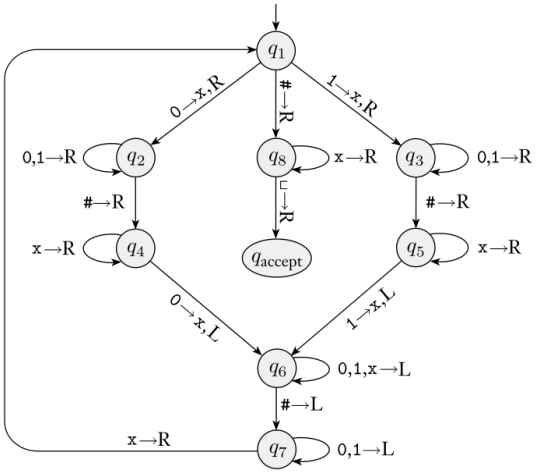


Figure: State diagram for TM M_1

TM computation

A TM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ computes as follows:

- Initially, M receives its input $w = w_1 w_2 ... w_n \in \Sigma^*$ on the leftmost n squares of the tape, and the rest of the tape is blank (i.e., filled with blank symbols)
- The head starts on the leftmost square of the tape. Note that Σ does not contain the blank symbol, so the first blank appearing on the tape marks the end of the input
- Once M has started, the computation proceeds according to the rules described by the transition function
- If M ever tries to move its head to the left off the left-hand end of the tape, the head stays in the same place for that move, even though the transition function indicates L
- The computation continues until it enters either the accept or reject states, at which point it halts. If neither occurs, M goes on forever.

TM computation

- As a TM computes, changes occur in the current state, the current tape contents, and the current head location
- A setting of these three items is called a *configuration* of the TM. Configurations often are represented in a special way
- For a state q and two strings u and v over the tape alphabet Γ, we write uqv for the configuration where the current state is q, the current tape contents is uv, and the current head location is the first symbol of v. The tape contains only blanks following the last symbol of v
- For example, $1011q_701111$ represents the configuration when the tape is 101101111, the current state is q_7, and the head is currently on the second 0

TM computation

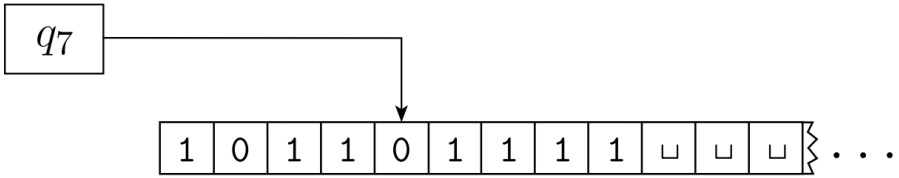


Figure: A TM with configuration $1011q_701111$

TM computation

Now we formalize our intuitive understanding of the way that a TM computes

- Say that configuration C_1 yields configuration C_2 if the TM can legally go from C_1 to C_2 in a single step
- Suppose that we have $a, b, c \in \Gamma$ and $u, v \in \Gamma^*$ and states q_i and q_j. In that case uaq_ibv and uq_jacv are two configurations
- We say that uaq_ibv yields uq_jacv if the transition function $\delta(q_i, b) = (q_j, c, L)$. That handles the case where the TM moves leftward
- For a rightward move, say that uaq_ibv yields $uacq_jv$ if the transition function $\delta(q_i, b) = (q_j, c, R)$
- Special cases occur when the head is at one of the ends of the configuration

TM computation

- The start configuration of M on input w is the configuration $q_0 w$, which indicates that the machine is in the start state q_0 with its head at the leftmost position on the tape
- In an accepting configuration, the state of the configuration is q_{accept}
- In a rejecting configuration, the state of the configuration is q_{reject}
- Accepting and rejecting configurations are *halting configurations* and do not yield further configurations
- A TM M accepts input w if a sequence of configurations $C_1, C_2, \ldots, C_k$ exists, where
  1. C_1 is the start configuration of M on input w
  2. each C_i yields C_{i+1}
  3. C_k is an accepting configuration

Recognizable and decidable

- The collection of strings that M accepts is the language of M or the language recognized by M, denoted $L(M)$
- When we start a TM on an input, three outcomes are possible. The machine may accept, reject, or loop. By loop we mean that the machine simply does not halt
- M can fail to accept an input by entering the q_{reject} state and rejecting, or by looping. Sometimes distinguishing a machine that is looping from one that is merely taking a long time is difficult. For this reason, we prefer Turing machines that halt on all inputs; such machines never loop.
- These machines are called *deciders* because they always make a decision to accept or reject. A decider that recognizes some language also is said to decide that language

Recognizable and decidable

Definition

Call a language Turing-recognizable if some Turing machine recognizes it

Definition

Call a language Turing-decidable or simply decidable if some Turing machine decides it

Examples

- Next, we study examples of decidable languages
- Every decidable language is Turing-recognizable
- As we did for FA and PDA, we can formally describe a particular TM by specifying each of its seven parts
- However, going to that level of detail can be cumbersome for almost all TM
- So, we will give only higher level descriptions because they are precise enough for our purposes and are much easier to understand

Example I

We describe M_2 that decides $A = \{0^{2^n} | n \geq 0\}$ the language consisting of all strings of 0s whose length is a power of 2

M_2 = On input string w:

1. Sweep left to right across the tape, crossing off every other 0
2. If in stage 1 the tape contained a single 0, *accept*
3. If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, *reject*
4. Return the head to the left-hand end of the tape
5. Go to stage 1

Example I

- Each iteration of stage 1 cuts the number of 0s in half
- As the machine sweeps across the tape in stage 1, it keeps track of whether the number of 0s seen is even or odd
- If that number is odd and greater than 1, the original number of 0s in the input could not have been a power of 2
- Therefore, the machine rejects in this instance. However, if the number of 0s seen is 1, the original number must have been a power of 2. So in this case, the machine accepts

Example I

Formal description of $M_2 = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{accept}, q_{reject}\}$
- $\Sigma = \{0\}$
- $\Gamma = \{0, x, \textvisiblespace\}$
- The start, accept and reject states are q_1, q_{accept}, and q_{reject}, respectively.
- δ is described in the next figure

Example I

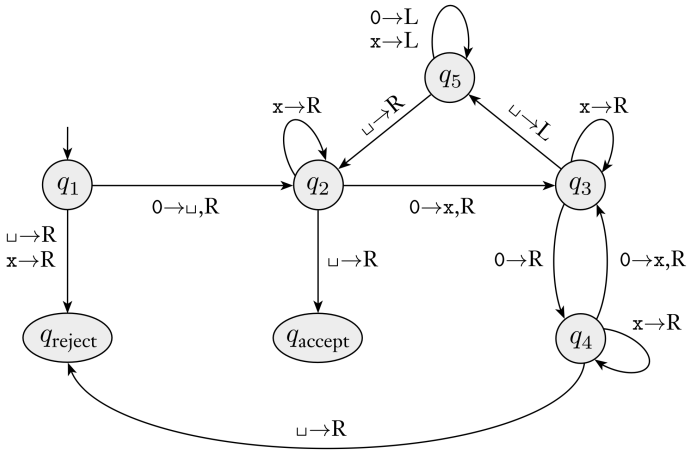


Figure: State diagram for Turing machine M_2

Example I

- The label $0 \to \sqcup, R$ appears on the transition from q_1 to q_2
- This label signifies that when in state q_1 with the head reading 0, the machine goes to state q_2, writes $\sqcup$ and moves the head to the right
- In terms of δ function we have: $\delta(q_1, 0) = (q_2, \sqcup, R)$
- For clarity we use the shorthand $0 \to R$ in the transition from q_3 to q_4, to mean that the machine moves to the right when reading 0 in state q_3 but doesn't alter the tape, so $\delta(q_3, 0) = (q_4, 0, R)$
- This machine begins by writing a blank symbol over the leftmost 0 on the tape so that it can find the left-hand end of the tape in stage 4

Example I

Below we have a sample run of this machine on input 0000. The starting configuration is $q_1 0000$. The sequence of configurations the machine enters appears as follows; read down the columns and left to right

$q_1 0000$
$\sqcup q_2 000$
$\sqcup \mathtt{x} q_3 00$
$\sqcup \mathtt{x} 0 q_4 0$
$\sqcup \mathtt{x} 0 \mathtt{x} q_3 \sqcup$
$\sqcup \mathtt{x} 0 q_5 \mathtt{x} \sqcup$
$\sqcup \mathtt{x} q_5 0 \mathtt{x} \sqcup$

$\sqcup q_5 \mathtt{x} 0 \mathtt{x} \sqcup$
$q_5 \sqcup \mathtt{x} 0 \mathtt{x} \sqcup$
$\sqcup q_2 \mathtt{x} 0 \mathtt{x} \sqcup$
$\sqcup \mathtt{x} q_2 0 \mathtt{x} \sqcup$
$\sqcup \mathtt{xx} q_3 \mathtt{x} \sqcup$
$\sqcup \mathtt{xxx} q_3 \sqcup$
$\sqcup \mathtt{xx} q_5 \mathtt{x} \sqcup$

$\sqcup \mathtt{x} q_5 \mathtt{xx} \sqcup$
$\sqcup q_5 \mathtt{xxx} \sqcup$
$q_5 \sqcup \mathtt{xxx} \sqcup$
$\sqcup q_2 \mathtt{xxx} \sqcup$
$\sqcup \mathtt{x} q_2 \mathtt{xx} \sqcup$
$\sqcup \mathtt{xx} q_2 \mathtt{x} \sqcup$
$\sqcup \mathtt{xxx} q_2 \sqcup$
$\sqcup \mathtt{xxx} \sqcup q_{\text{accept}}$

Section 6

Variants of Turing Machines

Variants of TMs

- There are many variants of TMs, including versions with multiple tapes or with nondeterminism
- They are called variants of the Turing machine model
- The original model and its reasonable variants all have the same power—they recognize the same class of languages

Multitape TMs

- A multitape Turing machine is like an ordinary TM with several tapes
- Each tape has its own head for reading and writing
- Initially the input appears on tape 1, and the others start out blank
- Transition function δ is change to allow reading, writing and moving on some or all of the tapes simultaneously

Multitape TMs

Formalizing we have:

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R\}^k$$

where k is the number of tapes and the expression

$$\delta(q_i, a_1, \ldots, a_k) = (q_j, b_1 \ \ldots, b_k, L, R, \ldots, L)$$

means that if the machine is in state q_i and heads 1 through k are reading symbols a_1 through a_k, the machine goes to state q_j, writes symbols b_1 through b_k and directs each head to move left or right, or to stay put, as specified.

Multitape TMs

Multitape TMs appear to be more powerful than ordinary TM, but we can show that they are equivalent in power. Recall that two machines are equivalent if they recognize the same language

Theorem

Every multitape Turing machine has an equivalent single-tape Turing machine

Multitape TMs

Proof.

We show how to convert a multitape TM M to an equivalent singletape TM S. The key idea is to show how to simulate M with S. Say that M has k tapes. Then S simulates the effect of k tapes by storing their information on its single tape. It uses the new symbol $\#$ as a delimiter to separate the contents of the different tapes. In addition to the contents of these tapes, S must keep track of the locations of the heads. It does so by writing a tape symbol with a dot above it (e.g. $\dot{a}$) to mark the place where the head on that tape would be. Image all as "virtual" tapes and heads. The "dotted" tape symbols are simply new symbols that have been added to the tape alphabet.

Example

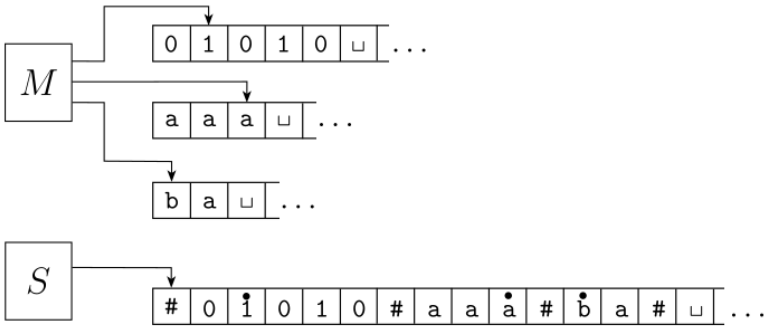


Figure: Representing three tapes with one

Multitape TMs

Proof.

$S =$ on input $w_1 \dots w_n$

- First S puts its tape into the format that represents all k tapes of M. The formatted tape contains $\#\dot{w}_1 w_2 \dots w_n \# \dot{\sqcup} \# \dot{\sqcup} \# \dots \#$
- To simulate a single move, S scans its tape from the first $\#$, which marks the left-hand end, to the $(k+1)$st $\#$, which marks the right-hand end, in order to determine the symbols under the virtual heads. Then S makes a second pass to update the tapes according to the way that M's transition function dictates.
- If at any point S moves one of the virtual heads to the right onto a $\#$, this action signifies that M has moved the corresponding head onto the previously unread blank portion of that tape. So S writes a blank symbol on this tape cell and shifts the tape contents, from this cell until the rightmost $\#$, one unit to the right. Then it continues the simulation as before

□